

AWS Lambda, API Gateway & KMS

1. Objective

To understand and implement AWS Lambda, Amazon API Gateway, and AWS Key Management Service (KMS), and demonstrate how they can work together to build a secure serverless application.

2. AWS Lambda

2.1 What is AWS Lambda?

AWS Lambda is a serverless, event-driven compute service that allows you to run code without managing servers.

AWS manages the underlying infrastructure, scaling, operating system, and server maintenance, while the user focuses on the application code.

2.2 Why do we use Lambda?

Lambda is used when we want to execute code without maintaining servers.

Common use cases:

- Serverless APIs
 - File processing
 - Image processing
 - Scheduled tasks
 - Automation
 - Event-driven applications
 - Backend services
 - Microservices
-

2.3 Lambda Function

A **Lambda function** is the code that AWS executes when the function is invoked.

Example:

```
def lambda_handler(event, context):  
    return {  
        "statusCode": 200,  
        "body": "Hello from Lambda"
```

}

2.4 Runtime

A **runtime** provides the environment required to execute Lambda code.

Examples:

- Python
- Node.js
- Java
- Go
- .NET

For this implementation, we can use **Python**.

2.5 Handler

The **handler is the entry point of a Lambda function**.

It tells Lambda which function to execute.

Example:

```
def lambda_handler(event, context):
```

Here:

lambda_handler → Handler

2.6 Event and Trigger

An **event** is the input data provided to Lambda.

A **trigger** is the service or event that causes Lambda to execute.

Example:

API Gateway

↓

Trigger

↓

Lambda

↓

Event

Common triggers include:

- API Gateway
 - S3
 - SQS
 - EventBridge
 - DynamoDB
-

2.7 Execution Role

An **IAM execution role** provides Lambda with permission to access AWS resources.

Example:

Lambda



IAM Execution Role



KMS Permission



KMS Key

For our implementation, the Lambda role will require appropriate KMS permissions.

2.8 Execution Environment

An execution environment provides the resources required to run a Lambda function, including the runtime, function code, dependencies, and temporary storage.

The environment may be reused for subsequent invocations.

2.9 Cold Start and Warm Start

Cold Start

A cold start occurs when Lambda needs to initialize a new execution environment before running the function.

Request



Initialize Environment



Start Runtime



Run Function

Warm Start

A warm invocation can reuse an existing execution environment.

Request



Existing Environment



Run Function

2.10 Scaling and Concurrency

Lambda automatically scales based on incoming requests.

Concurrency means the number of Lambda executions running at the same time.

Example:

Request 1 → Lambda Execution

Request 2 → Lambda Execution

Request 3 → Lambda Execution

Lambda can handle multiple concurrent executions without manually creating additional servers.

2.11 CloudWatch Monitoring

Lambda integrates with Amazon CloudWatch for monitoring.

Important metrics include:

- Invocations
- Errors
- Duration
- Throttles
- Concurrent executions

Lambda execution logs can also be viewed in CloudWatch Logs.

2.12 Lambda Architecture

Client/Event



Lambda Trigger



Lambda Function



Application Logic



Response

3. Amazon API Gateway

3.1 What is API Gateway?

Amazon API Gateway is a managed AWS service used to create, publish, secure, monitor, and manage APIs.

It can act as the entry point for clients to access backend services such as Lambda.

3.2 Why do we use API Gateway?

API Gateway is commonly used to:

- Expose Lambda functions through HTTP APIs
 - Create REST APIs
 - Manage API requests
 - Apply authorization
 - Control traffic through throttling
 - Monitor API activity
-

3.3 API Gateway Architecture

For our implementation:

Client



API Gateway



Lambda



Response

3.4 API Endpoint

An **API endpoint** is the **URL** through which a client accesses an **API**.

Example:

`https://xxxxx.execute-api.us-east-1.amazonaws.com`

A route can be:

`GET /hello`

3.5 Route

A route defines the **HTTP method and path** that API Gateway should handle.

Example:

`GET /hello`

Here:

`GET` → HTTP method

`/hello` → Path

3.6 HTTP Methods

Method Purpose

GET	Retrieve data
POST	Create/send data
PUT	Update/replace data
PATCH	Partially update data
DELETE	Delete data

For our simple implementation:

`GET /hello`

3.7 Integration

An **integration** connects an API Gateway route to a backend service.

For our project:

API Gateway



Lambda Integration



Lambda Function

When a client calls the API, API Gateway invokes Lambda.

3.8 Authentication and Authorization

Authentication verifies who the user is.

Authorization determines what the authenticated user is allowed to access.

API Gateway supports authorization mechanisms such as:

- IAM authorization
 - JWT authorization
 - Lambda authorizers
-

3.9 Throttling

Throttling controls the number of API requests allowed over a period.

It helps protect backend services from excessive traffic.

Large number of requests



API Gateway



Throttling

3.10 CORS

CORS stands for Cross-Origin Resource Sharing.

It controls whether a browser application from one origin can make requests to an API on another origin.

For example:

React Application



API Gateway

CORS may need to be configured when the frontend and API have different origins.

3.11 API Gateway Monitoring

API Gateway can be monitored using CloudWatch.

Important metrics include:

- Request count
 - Latency
 - 4xx errors
 - 5xx errors
 - Integration latency
-

4. AWS KMS

4.1 What is AWS KMS?

AWS Key Management Service (KMS) is a managed service used to create and control cryptographic keys used to protect data.

It can be integrated with many AWS services such as:

- S3
 - EBS
 - RDS
 - Lambda
 - Secrets Manager
-

4.2 Why do we use KMS?

KMS is used to:

- Create encryption keys
- Control access to keys
- Encrypt and decrypt data

- Protect data at rest
 - Integrate encryption with AWS services
 - Audit key usage
-

4.3 Encryption

Encryption converts readable data (**plaintext**) into protected data (**ciphertext**).

Plaintext



Encryption



Ciphertext

4.4 Decryption

Decryption converts ciphertext back into the original readable data.

Ciphertext



Decryption



Plaintext

4.5 KMS Key

A **KMS key** is a cryptographic key managed through AWS KMS.

The key can be used for supported encryption and decryption operations.

Example:

Application



KMS Key



Encryption



Encrypted Data

4.6 Symmetric vs Asymmetric Keys

Symmetric Key

The same cryptographic key is used for supported encryption and decryption operations.

Same Key

/ \

Encrypt Decrypt

Symmetric KMS keys are commonly used for AWS data-at-rest encryption.

Asymmetric Key

Uses a pair of keys:

Public Key

Private Key

They are used for supported encryption/decryption or signing/verification use cases.

4.7 AWS Managed vs Customer Managed Keys

AWS Managed Key

AWS manages the key for a supported AWS service within your account.

Examples may include:

aws/s3

aws/ebs

aws/rds

Customer Managed Key

The customer creates and manages the key.

The customer can control:

- Key policy
- Permissions
- Rotation settings
- Enable/disable state
- Deletion scheduling

For our implementation, we can create a **customer managed symmetric KMS key**.

4.8 Key Policy

A **KMS key policy** controls who can access and use the KMS key and which operations they can perform.

Examples:

kms:Encrypt

kms:Decrypt

kms:GenerateDataKey

kms:DescribeKey

4.9 IAM and KMS

IAM controls what the Lambda execution role is allowed to request, while the KMS key policy also controls access to the key.

Example:

Lambda



IAM Execution Role



KMS Permissions



KMS Key

For our implementation, the Lambda role will receive only the KMS permissions required by the function.

4.10 Key Rotation

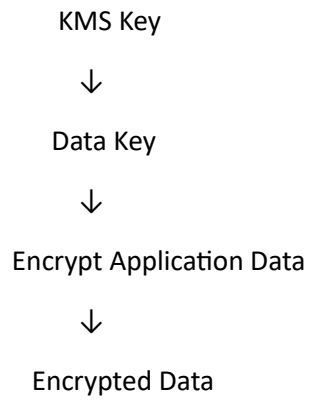
Key rotation changes the cryptographic key material used by a KMS key according to KMS's supported rotation mechanism.

For supported customer-managed symmetric keys, automatic rotation can be configured.

Rotation does not mean previously encrypted data automatically becomes inaccessible.

4.11 Envelope Encryption

Envelope encryption uses a data key to encrypt the actual data, while a KMS key protects the data key.



This is an important advanced KMS concept.

4.12 CloudTrail

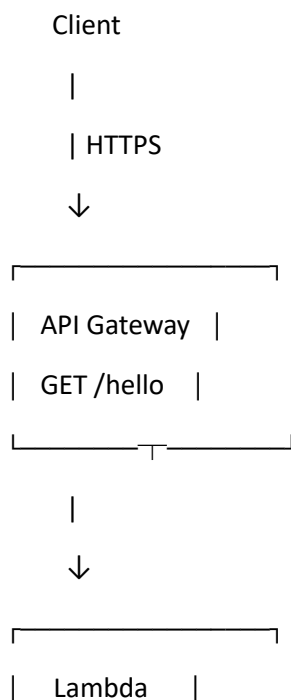
AWS CloudTrail can be used to audit KMS API activity.

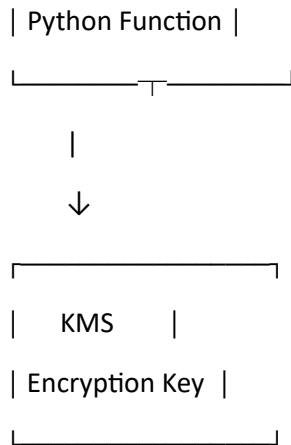
It can help identify:

- Who performed an operation
 - When it happened
 - Which KMS API was called
 - Which key was involved
-

5. End-to-End Implementation

The three services can be connected as follows:





Flow

1. Client sends an HTTP request.
2. API Gateway receives the request.
3. API Gateway invokes Lambda.
4. Lambda executes the application logic.
5. Lambda uses KMS when encryption/decryption is required.
6. Lambda returns the response.
7. API Gateway sends the response to the client.

6. Implementation Steps

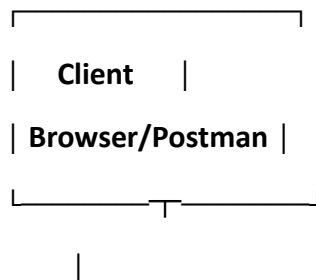
AWS Lambda + API Gateway + KMS Implementation

1. Objective

To implement a serverless API using Amazon API Gateway and AWS Lambda, with AWS KMS used for encryption and decryption.

2. Architecture

High-Level Architecture



HTTPS Request



API Gateway	
GET /hello	



Lambda Integration



AWS Lambda	
aws-learning-lambda	
Python 3.14	



KMS API Request

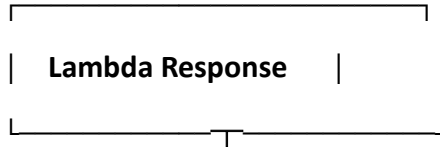


AWS KMS	
Customer Managed Key	
alias/aws-learning-key	



Encrypt / Decrypt

|



|



API Gateway

|



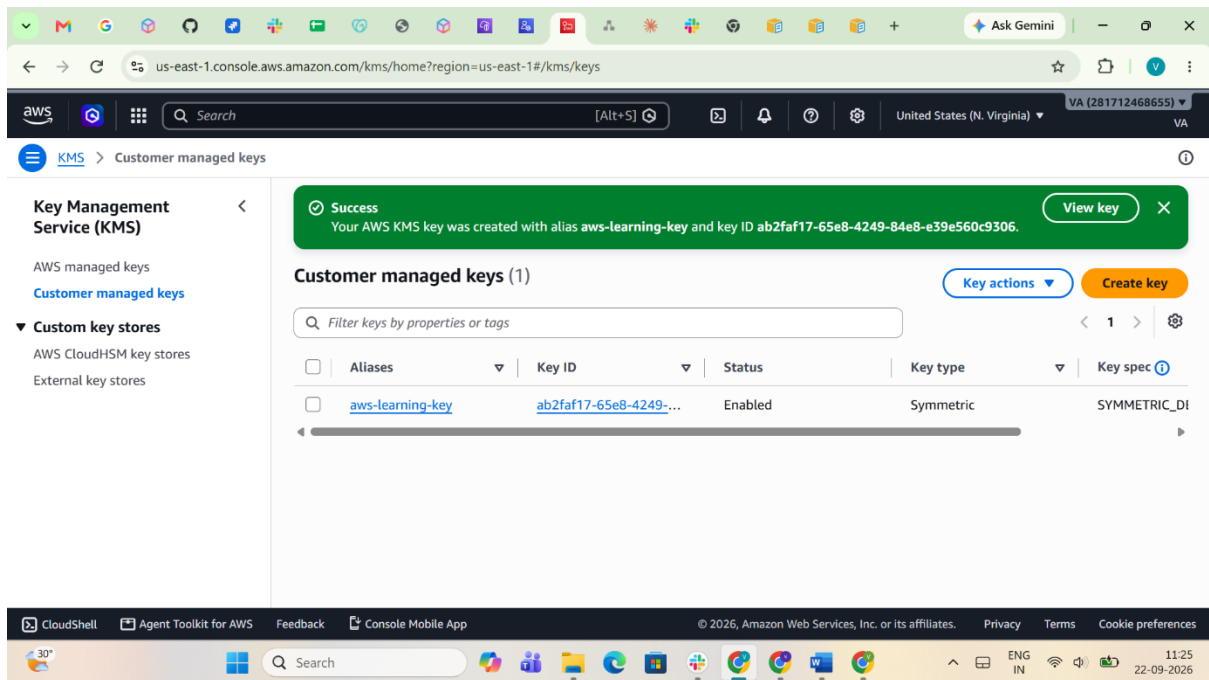
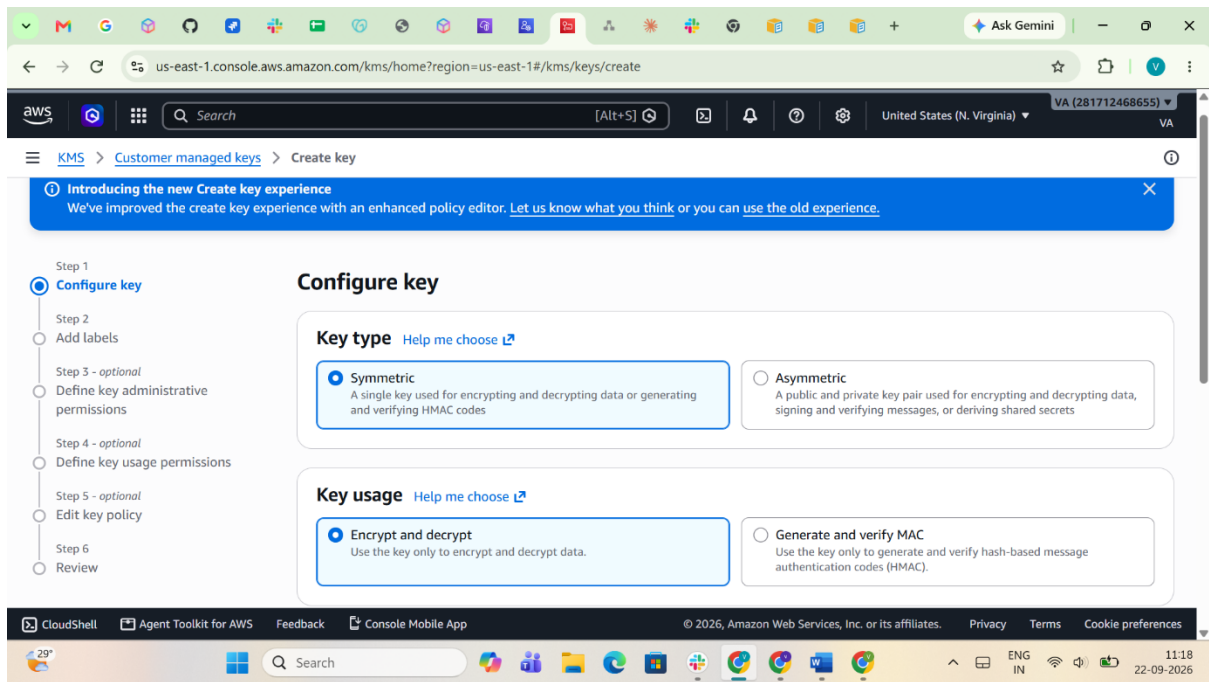
Client

3. Implementation

Step 1: Create KMS Key

1. Open AWS KMS.
2. Navigate to Customer managed keys.
3. Click Create key.
4. Select:
 - Key type: Symmetric
 - Key usage: Encrypt and decrypt
5. Add alias: aws-learning-key
6. Select the IAM user as Key Administrator.
7. Create the key.

Screenshot: KMS key configuration/details.



Step 2: Verify KMS Key

Verify the following:

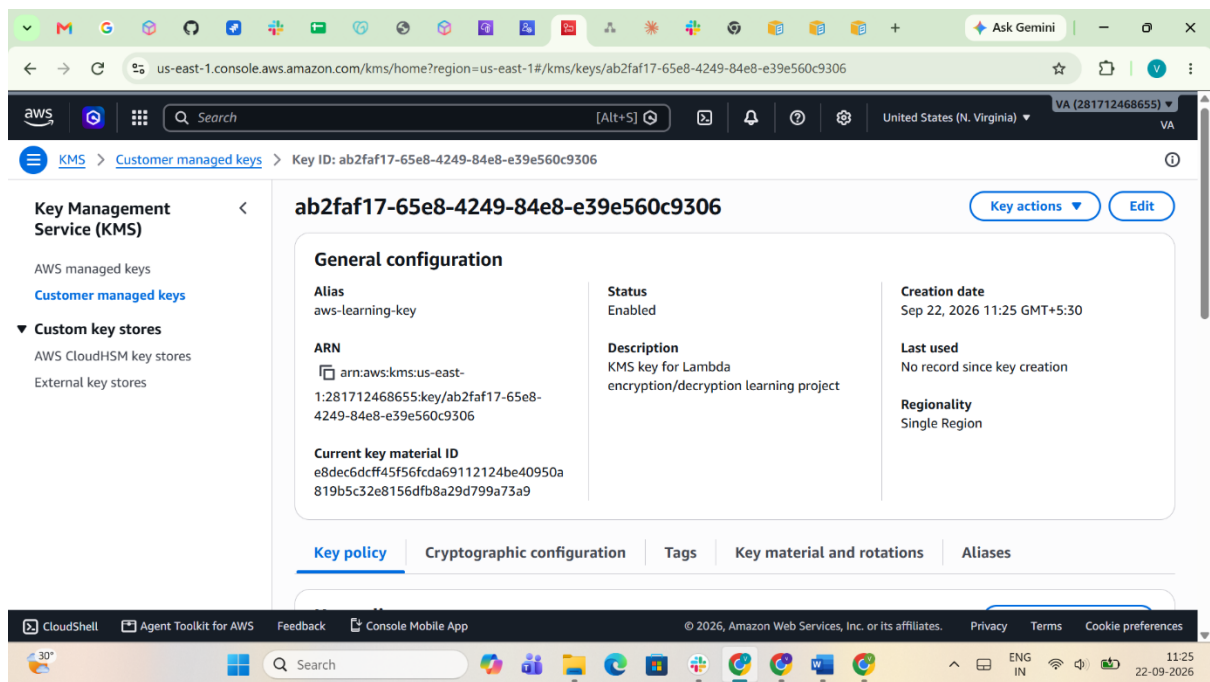
Key Type: Symmetric

Key Usage: Encrypt and decrypt

Alias: aws-learning-key

Status: Enabled

Screenshot: KMS key details.



Step 3: Create Lambda Function

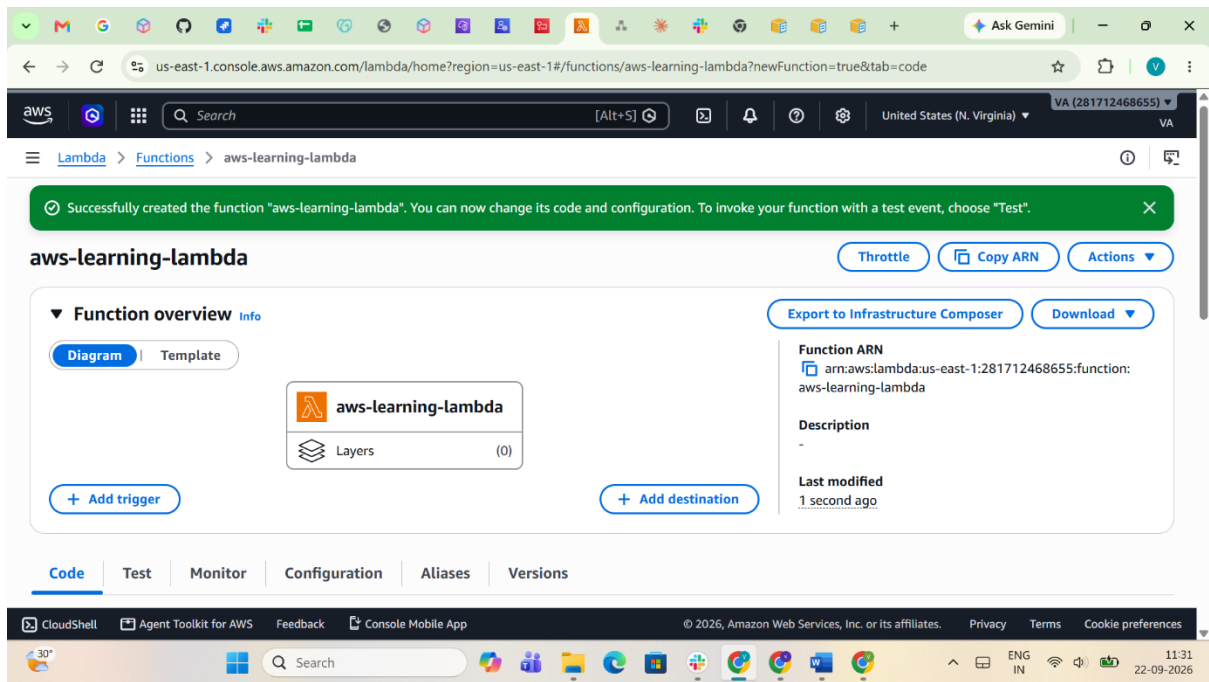
1. Open AWS Lambda.
2. Click Create function.
3. Select Author from scratch.
4. Enter:

Function name: aws-learning-lambda

Runtime: Python 3.14

5. Create a new execution role with basic Lambda permissions.
6. Create the function.

Screenshot: Lambda function configuration.



Step 4: Configure Lambda Code

```
import json
```

```
import boto3
```

```
import base64
```

```
kms = boto3.client('kms')
```

```
KMS_KEY_ID = "ab2faf17-65e8-4249-84e8-e39e560c9306"
```

```
def lambda_handler(event, context):
```

```
    plaintext = "Hello from AWS Lambda with KMS"
```

```
    # Encrypt
```

```
    encrypt_response = kms.encrypt(
```

```
        KeyId=KMS_KEY_ID,
```

```
        Plaintext=plaintext.encode("utf-8")
```

```
)
```

```
ciphertext = encrypt_response["CiphertextBlob"]
```

```
# Decrypt
```

```
decrypt_response = kms.decrypt(
```

```
    CiphertextBlob=ciphertext
```

```
)
```

```
decrypted_text = decrypt_response["Plaintext"].decode("utf-8")
```

```
return {
```

```
    "statusCode": 200,
```

```
    "body": json.dumps({
```

```
        "message": "Lambda and KMS integration successful",
```

```
        "original_text": plaintext,
```

```
        "decrypted_text": decrypted_text
```

```
    })
```

```
}
```

Add the Lambda code to:

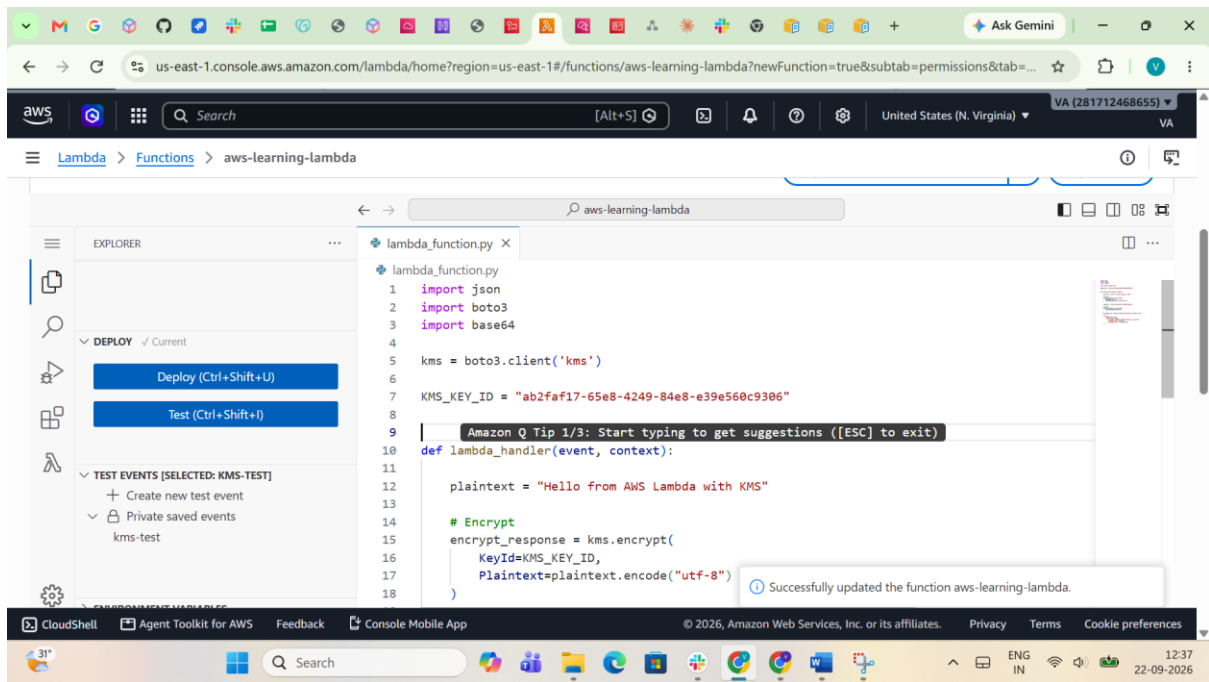
- Receive the test event.
- Encrypt data using KMS.
- Decrypt the encrypted data using KMS.
- Return the decrypted result.

Use:

```
KMS_KEY_ID = "aws-learning-key"
```

Deploy the function.

Screenshot: Lambda code.



Step 5: Configure Lambda IAM Permissions

1. Open the Lambda function.
2. Go to Configuration → Permissions.
3. Open the Lambda Execution Role.
4. Add an inline IAM policy.
5. Allow:

kms:Encrypt

kms:Decrypt

6. Restrict the resource to your specific KMS key.

Screenshot: Lambda execution role and KMS permissions.

Step 6: Configure KMS Key User

1. Open the KMS customer-managed key.
2. Open Key policy.
3. Add the Lambda execution role as a Key User, if required by your key policy configuration.
4. Allow the Lambda role to use the key for encryption/decryption.

The permission flow is:

Lambda



Lambda Execution Role

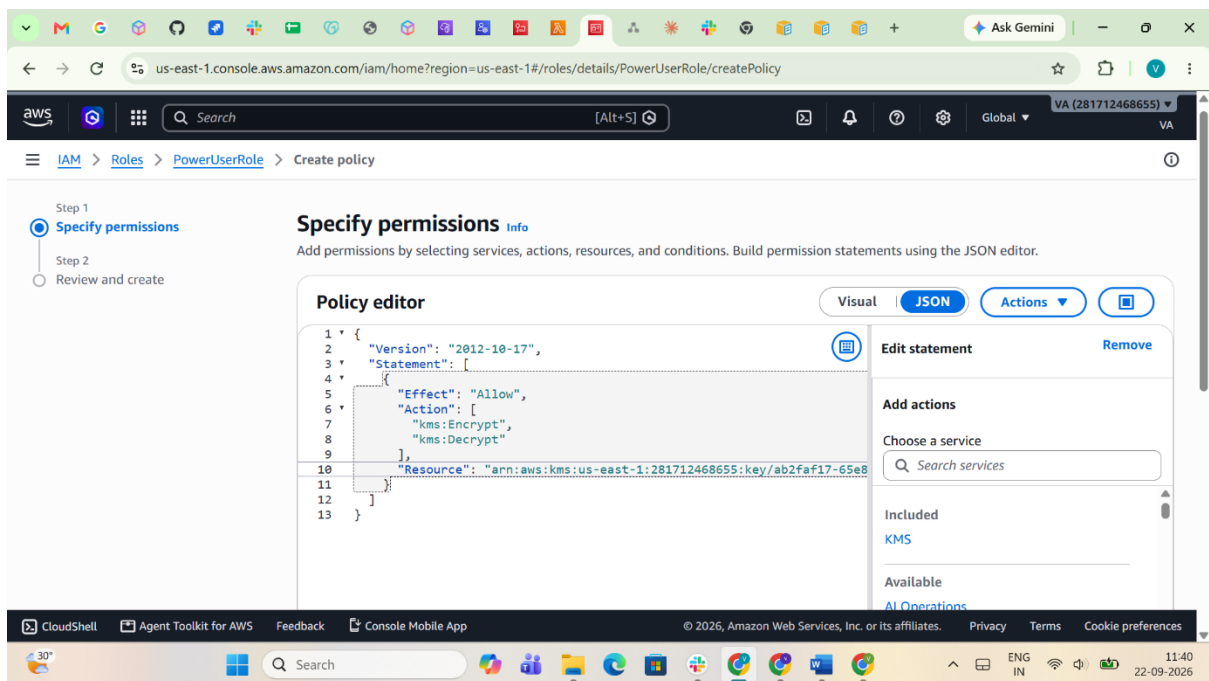


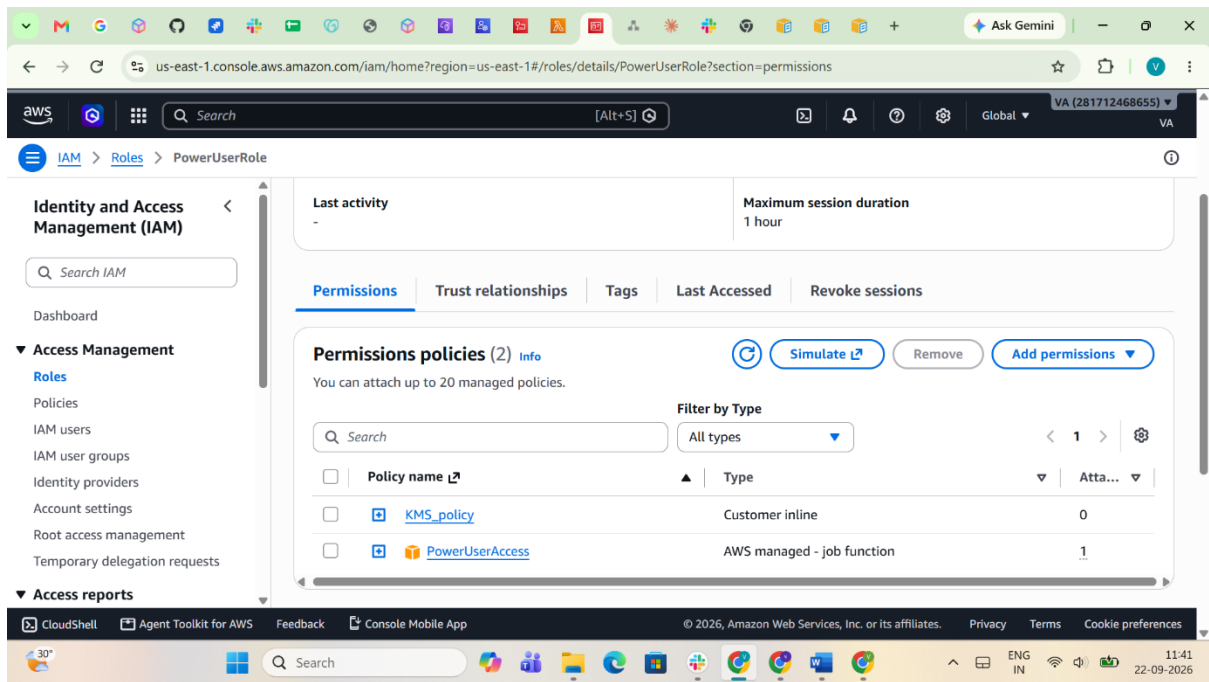
kms:Encrypt / kms:Decrypt



KMS Key

Screenshot: KMS key policy / Key users.





Step 7: Test Lambda

1. Open the Lambda function.
2. Select Test.
3. Create a new test event.
4. Use:

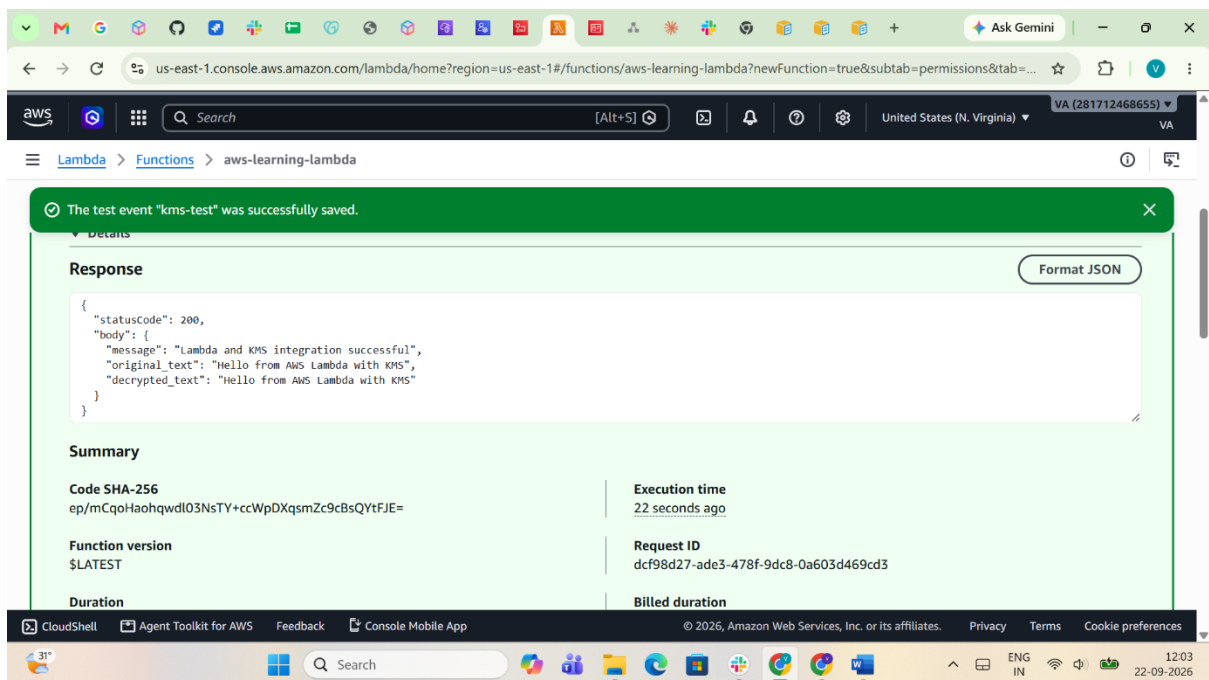
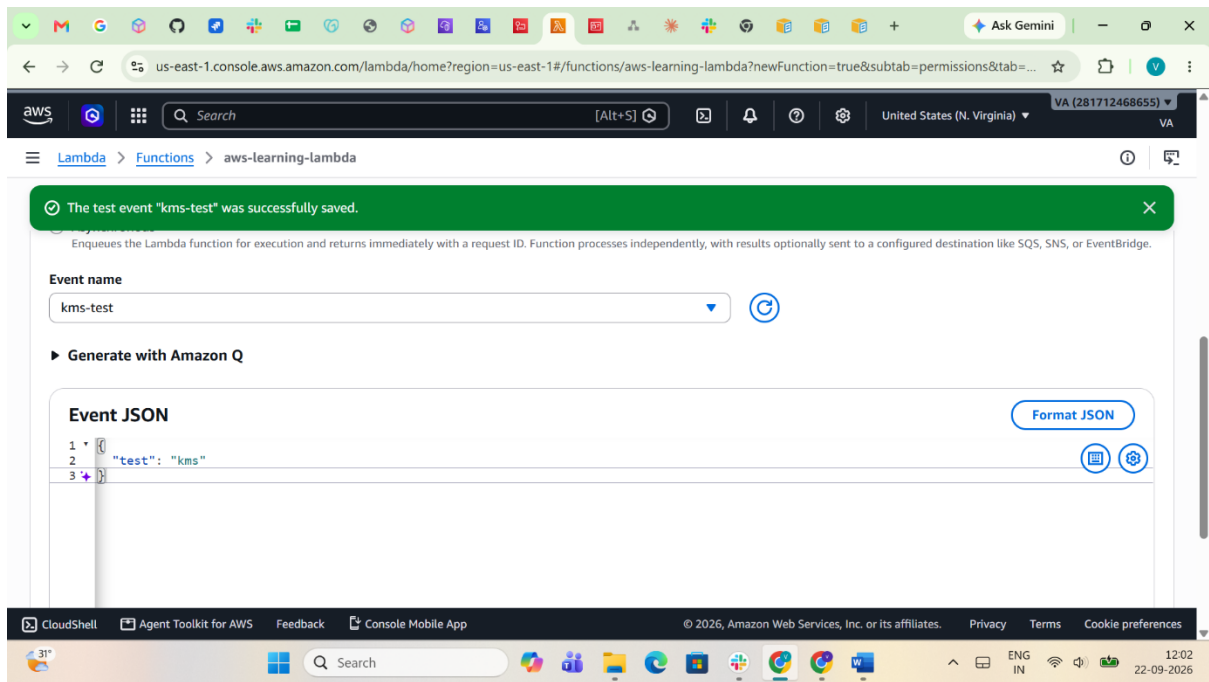
```
{  
  "test": "kms"  
}
```

5. Select Synchronous invocation.
6. Save and click Test.
7. Verify that encryption and decryption are successful.

Expected result:

Lambda and KMS integration successful

Screenshot: Successful Lambda test result.



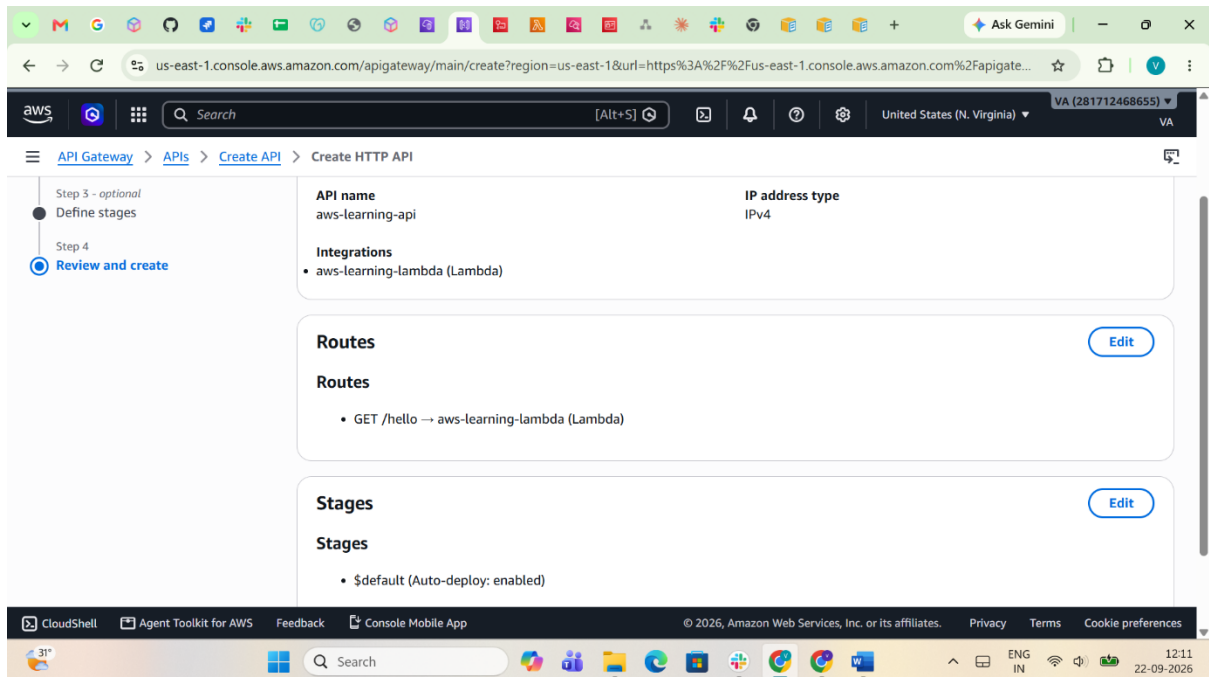
4. API Gateway Implementation

Step 8: Create HTTP API

1. Open Amazon API Gateway.
2. Select HTTP API.
3. Click Build.
4. Enter:

API name: aws-learning-api

Screenshot: API Gateway creation.



Step 9: Configure Lambda Integration

Select:

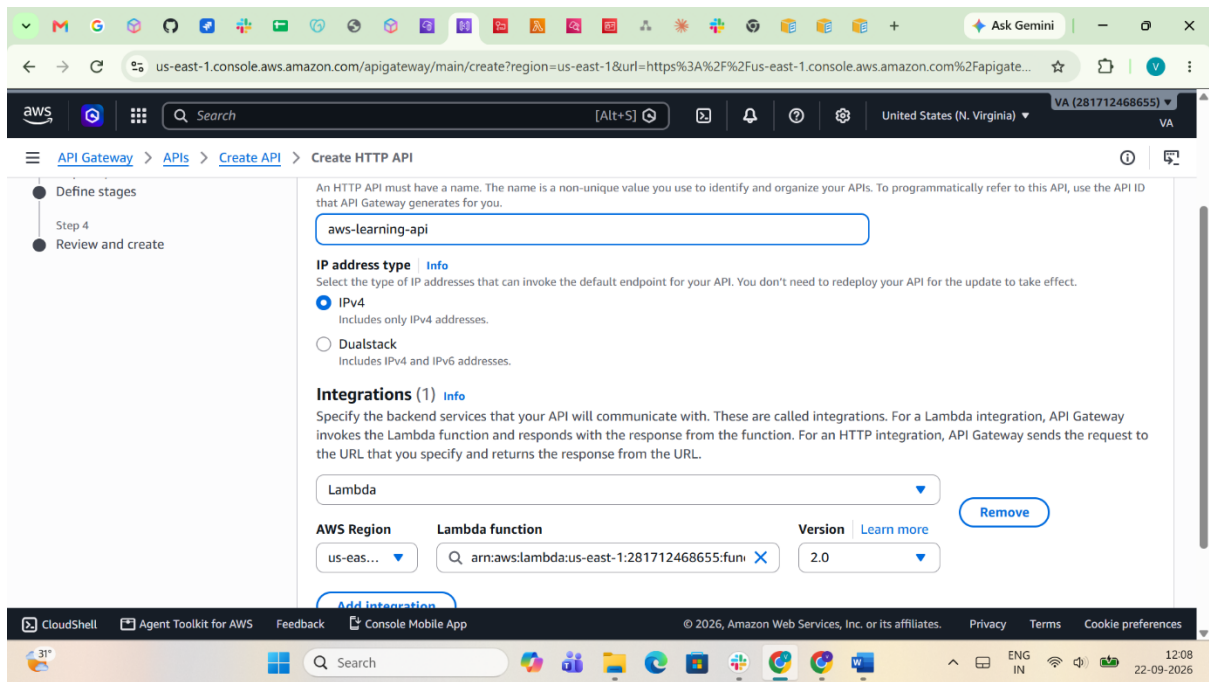
Integration:

Lambda

Lambda function:

aws-learning-lambda

Screenshot: Lambda integration.



Step 10: Configure API Route

Create the route:

Method: GET

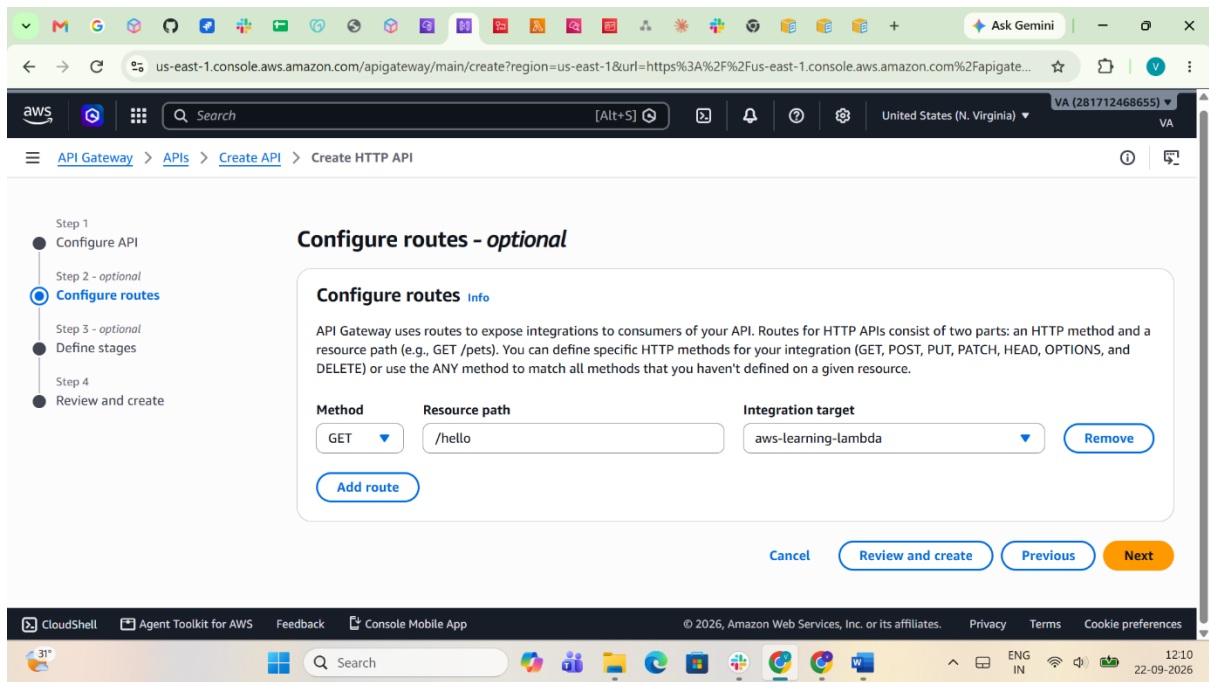
Resource path: /hello

Integration: aws-learning-lambda

Therefore:

GET /hello

Screenshot: API Gateway route configuration.



Step 11: Configure Stage

Use:

Stage:

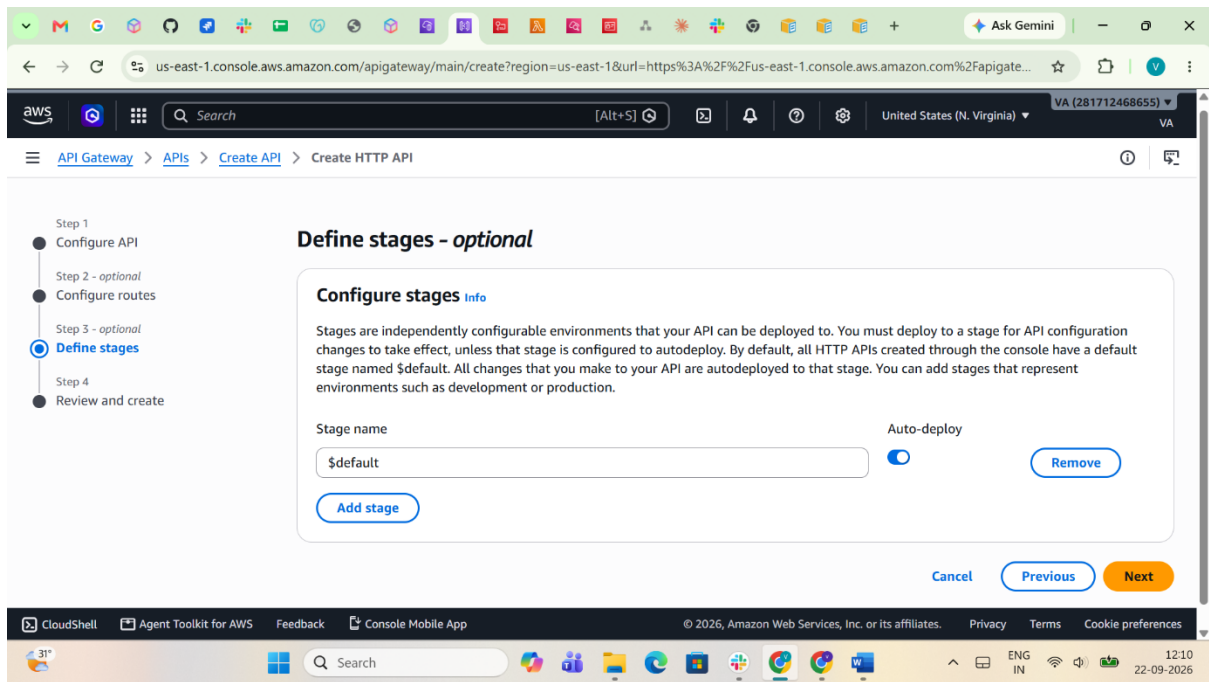
\$default

Auto-deploy:

Enabled

Because Auto-deploy is enabled, route/integration changes are automatically deployed.

Screenshot: \$default stage configuration.



Step 12: Obtain Invoke URL

Open:

API Gateway → aws-learning-api → Stages → \$default

Copy the Invoke URL.

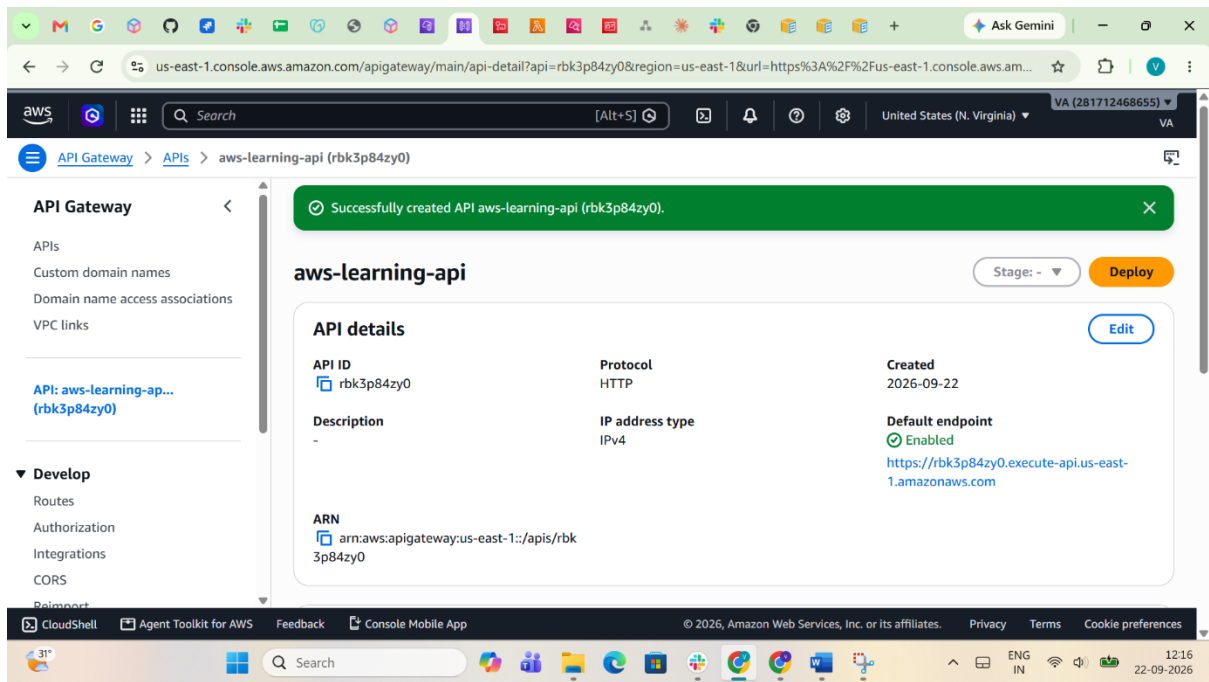
Your API endpoint will be:

<Invoke-URL>/hello

Example:

https://xxxxxxx.execute-api.us-east-1.amazonaws.com/hello

Screenshot: Invoke URL.



5. Test Complete Architecture

Open the API endpoint in a browser or Postman:

GET <Invoke-URL>/hello

The request flow is:

Client



API Gateway



GET /hello



Lambda



KMS Encrypt



KMS Decrypt

↓

Lambda Response

↓

API Gateway

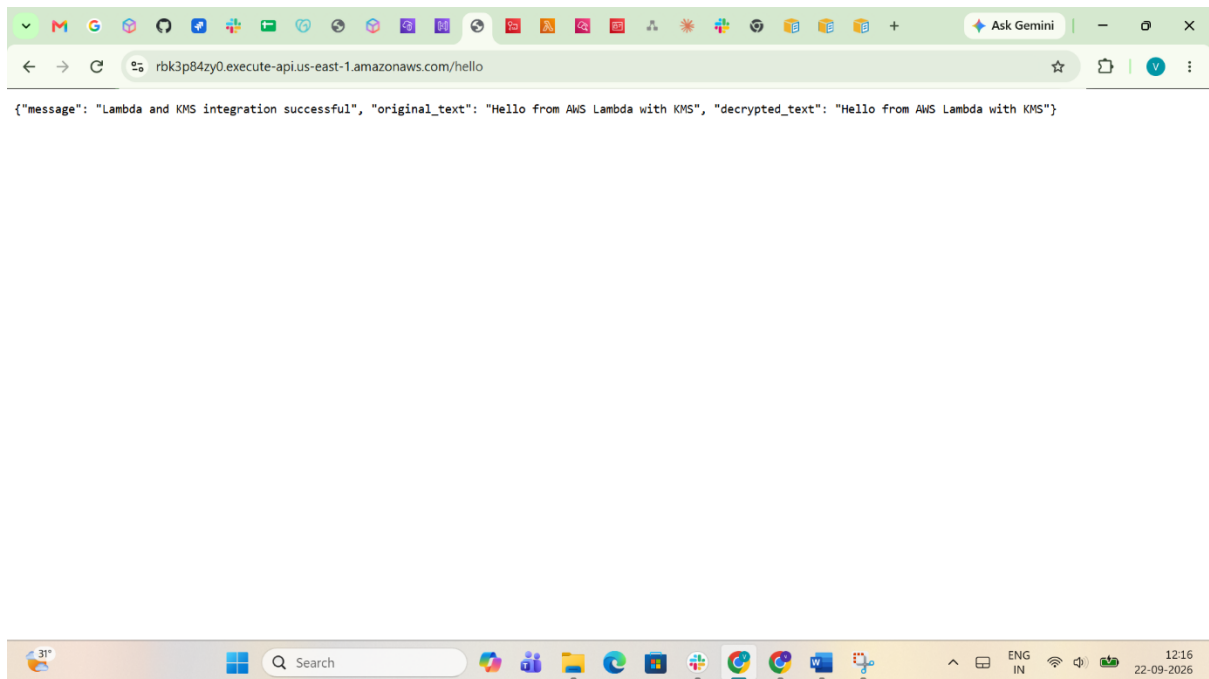
↓

Client

Expected response:

```
{  
  "message": "Lambda and KMS integration successful",  
  "original_text": "Hello from AWS Lambda with KMS",  
  "decrypted_text": "Hello from AWS Lambda with KMS"  
}
```

Screenshot: Successful API response.



6. Monitoring

Lambda

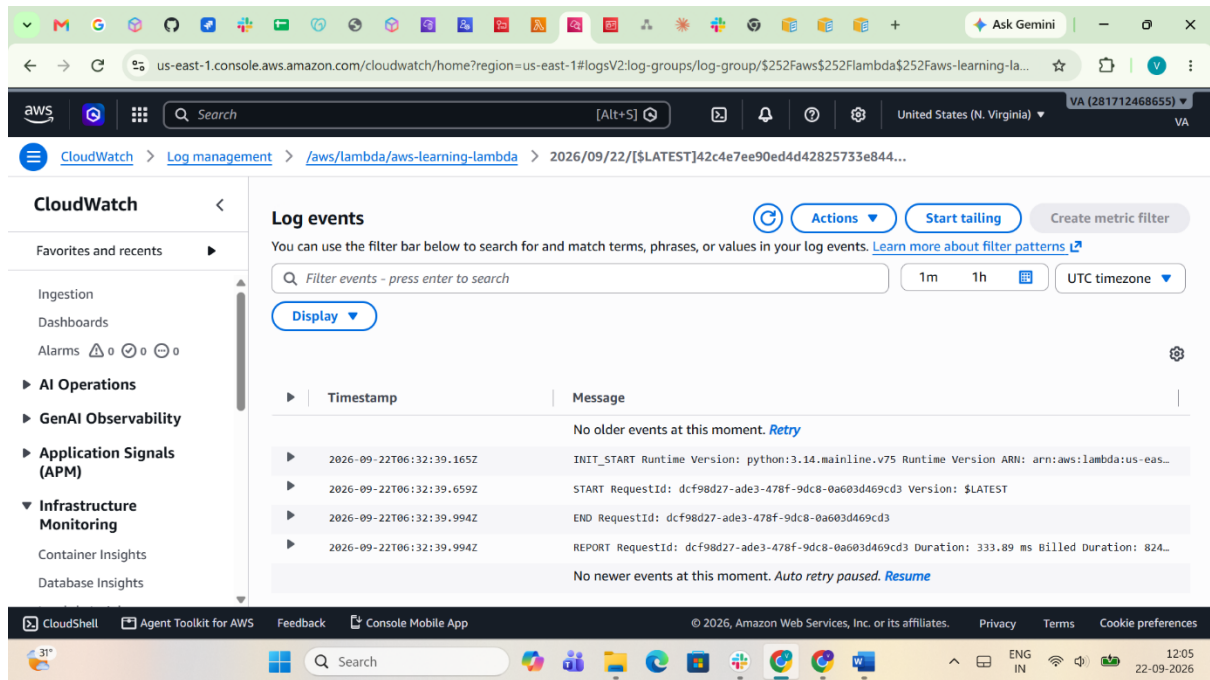
Verify Lambda execution through:

Lambda → Monitor → CloudWatch Logs

Check:

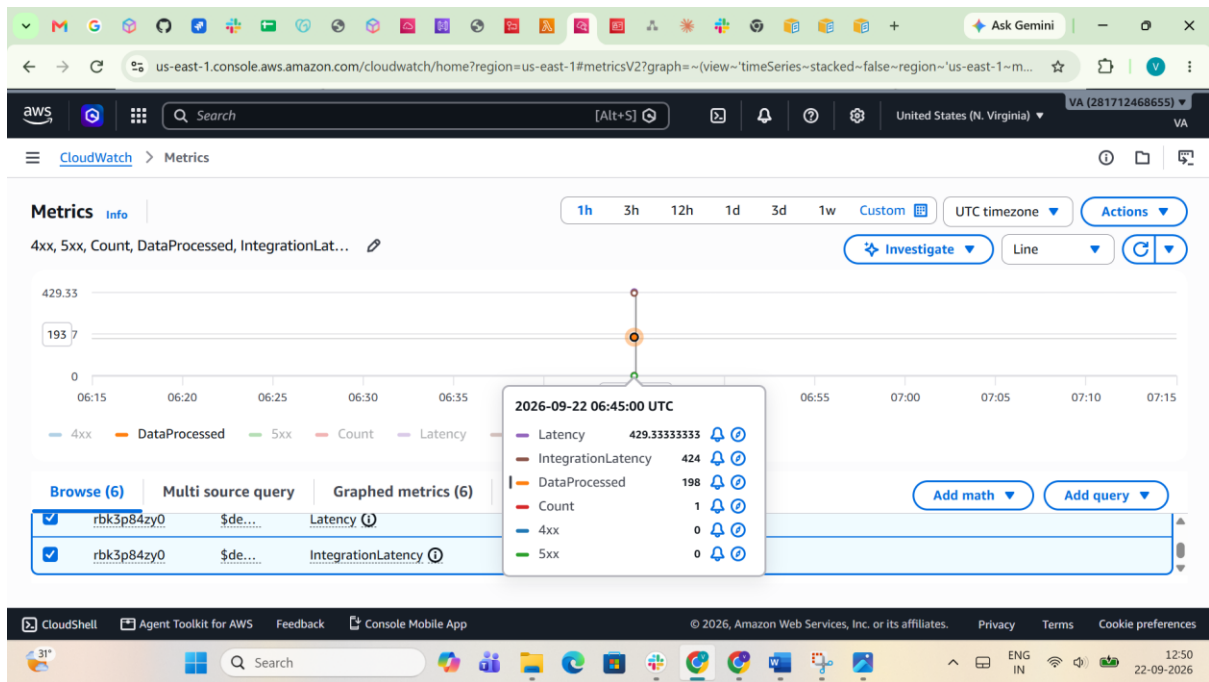
- Invocations
- Duration
- Errors
- Logs

Screenshot: CloudWatch Lambda logs.



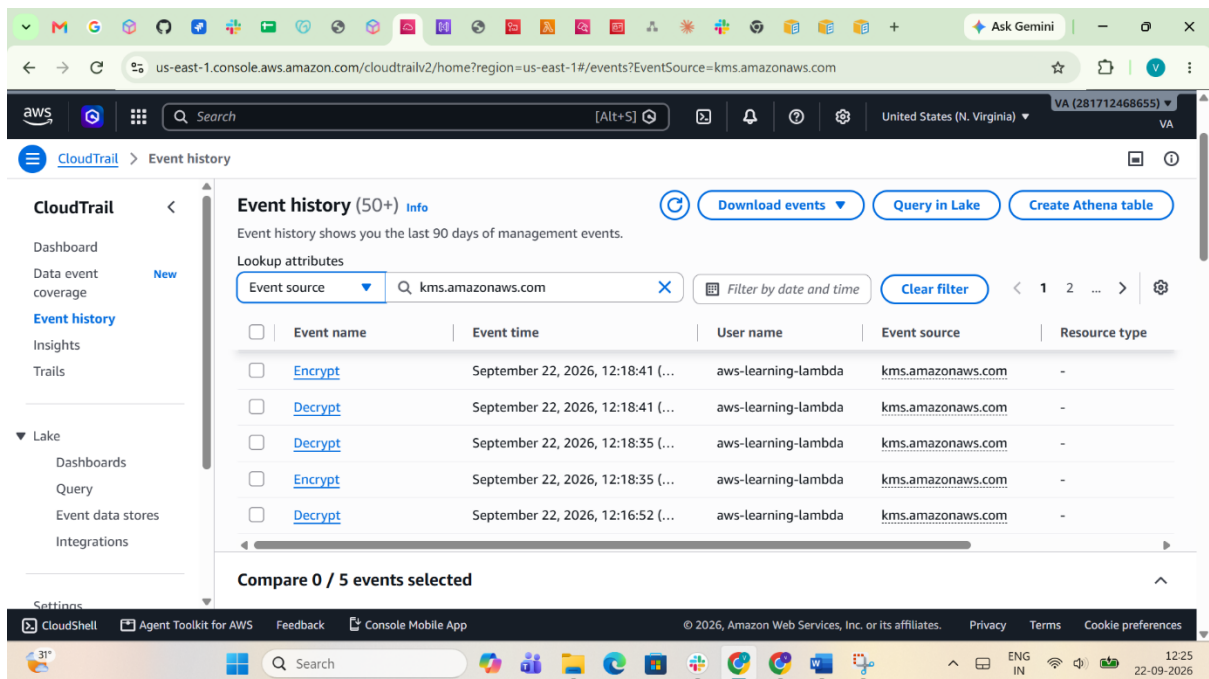
API Gateway

Verify API activity through API Gateway monitoring/CloudWatch where configured.



KMS

KMS API activity can be audited using AWS CloudTrail.

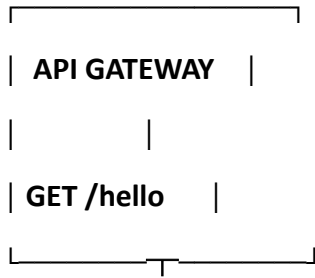


7. Final Implementation Architecture

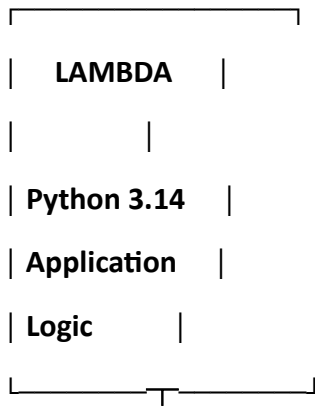
CLIENT

|

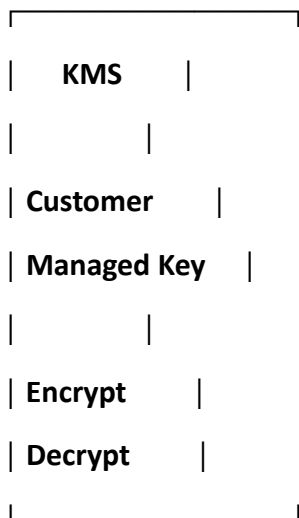
| HTTPS



Integration



KMS API

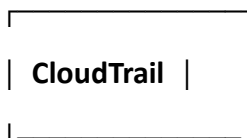


Monitoring



Lambda Logs

Auditing



KMS API

8. Observations

Lambda

The Lambda function was successfully created and executed without provisioning or managing an EC2 server. The function execution and logs were verified using CloudWatch.

API Gateway

An HTTP API was successfully created and integrated with the Lambda function. The API endpoint successfully invoked Lambda and returned the expected response.

KMS

A customer-managed symmetric KMS key was created and configured with the required permissions. The key was used for encryption/decryption operations.

Overall

API Gateway successfully provided the HTTP interface, Lambda handled the application logic, and KMS provided cryptographic key management for protecting data.

9. Conclusion

The implementation provided practical knowledge of AWS serverless computing, API management, and encryption. AWS Lambda was used to execute application code without managing servers, API Gateway was used to expose the Lambda function through an HTTP API, and AWS KMS was used to

manage encryption keys and protect application data. The integration demonstrated how these AWS services can be combined to build a serverless and secure application.